KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES - SYLLABUS

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

2026

Year:

I

Semester:

I

Unit:

I

Syllabus

Unit I - HyperText Markup Language (HTML5)

1. Web Technologies Overview
2. HTML5 Document Structure
3. Semantic Elements
4. Text, Hyperlinks and Navigation
5. Images and Media
6. Forms and Validation
7. Tables
8. Accessibility and SEO
9. Browser DevTools and Emmet

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM

Batch:

2026

Year:

I

Semester:

I

Unit:

I

Topic:

Web Design - Overview

---

1. Web Design

Explanation: Web Design is the process of planning, designing and organizing content for display on the Internet. It combines both visual (UI) and functional (UX) aspects. HTML provides structure, CSS provides style and JavaScript provides behavior. Web Design focuses on layout, color, fonts, navigation and responsiveness.

Syntax / Structure Concept:

```
Website = HTML (Structure) + CSS (Presentation) + JavaScript (Behavior)
        + Content + Navigation + Design Principles
```

Example: Portfolio website of a student, e-commerce site like Amazon, college website.

Example Explanation: A college website needs header with logo, nav menu (Home, About, Courses), main content area for course details, sidebar for notices and footer for contact. This planning is part of web design overview.

---

2. Internet

Explanation: Internet is a global system of interconnected computer networks that use TCP/IP protocol to communicate. It is decentralized and not owned by any single organization. It originated from ARPANET (1969). It provides services like Email, FTP, WWW, Video Conferencing.

Features / Syntax of Working:

- Uses Packet Switching
- Protocol: TCP/IP
- No central owner
- Global unique IP address for each device

Example: You connect laptop via ISP -> ISP connects to backbone network -> Accesses server hosted in USA.

Example Explanation: When you type google.com, your request goes through your ISP's network, then through multiple routers across the world via TCP/IP, reaches Google's server, and response comes back same path. This global network is Internet.

---

3. WWW (World Wide Web)

Explanation: WWW is an information system of interlinked hypertext documents accessed via Internet. Invented by Tim Berners-Lee at CERN in 1989. It uses HTML to create pages, URL to address pages and HTTP to transfer pages.

Syntax / Components:

```
WWW = Web Pages (HTML) + URL (Address) + HTTP/HTTPS (Protocol) + Hyperlinks
```

Example: `https://www.wikipedia.org/wiki/Internet` - This is a WWW resource. Wikipedia pages linked to each other via hyperlinks.

Example Explanation: WWW is like a library of documents stored on Internet servers. Each page is linked to another via hyperlink. You use browser to access these pages. WWW runs ON Internet.

---

4. Internet vs WWW - Difference Table

Explanation: Internet is infrastructure (hardware + software network). WWW is a service that runs on that infrastructure.

Syntax (Tabular Difference):

Point	Internet	WWW
What	Network of networks	Collection of web pages & info
Based On	TCP/IP	HTTP, HTML, URL
Invented	Vint Cerf & Bob Kahn (ARPANET)	Tim Berners-Lee (1989)

Point	Internet	WWW
Example	Cables, Routers, Servers	google.com, wikipedia.org

Example: You can have Internet without WWW (like using only Email via Internet). But you cannot have WWW without Internet.

Example Explanation: Think Internet = Road, WWW = Cars (websites) running on road. Road is required for cars to run.

---

5. Web Browser

Explanation: Web Browser is an application software used to access and view websites on WWW. It sends HTTP request to server and renders (converts) HTML/CSS/JS files into visual page.

Syntax / Working Steps:

```
1. User enters URL in Browser
2. DNS converts Domain to IP
3. Browser → HTTP Request → Server
4. Server → HTTP Response (HTML/CSS/JS) → Browser
5. Browser Parses & Renders Page
```

Example: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera, Brave.

Program / Usage Example:

```
<!-- User types in Browser address bar -->
https://www.example.com/index.html
<!-- Browser does the request and displays the file content visually -->
```

Example Explanation: Browser is like a translator. Server speaks in HTML/CSS code, user understands visual page. Browser translates code to visual page. It also stores History, Cache, Cookies.

Browser vs Search Engine:

Browser	Search Engine
Software to view websites	Website to search information
Ex: Chrome, Firefox	Ex: Google, Bing

---

6. Web Server, Domain, URL, Hosting, DNS

A) Web Server

Explanation: Web Server is a computer/software that stores website files and serves them to client browsers on request. Always online.

Syntax / Example: Apache, Nginx. Server address: `192.168.1.1`

Example: When you request `amazon.com`, Amazon's web server finds `index.html` and sends to your browser.

Example Explanation: Web server is like a waiter in restaurant. You (client) order (request) a dish (web page), waiter (server) brings it from kitchen (storage).

B) Domain Name

Explanation: Domain is human readable name that replaces complex IP address.

Syntax Structure:

```
www.example.com
 |       |     |
subdomain + SLD + TLD
www = World Wide Web, amazon = Second Level Domain, .com = Top Level Domain
```

Example: `google.com`, `abccollege.edu.in`

Example Explanation: Instead of remembering `142.250.195.46` (Google IP), we remember `google.com`. Easy for humans.

C) URL

Explanation: URL is complete address of a resource on web.

Syntax:

```
https://www.example.com:443/products/index.html?id=5#section
|       |                  |         |            |      |
Protocol  Domain            Port    Path         Query  Fragment
```

Example: `https://www.abccollege.edu.in:443/courses/bca.html`

Example Explanation: Protocol https tells secure transfer, domain tells which server, path tells which file inside server, query `?id=5` sends data and fragment `#section` jumps to section.

D) Web Hosting

Explanation: Web Hosting is service of storing website files on a web server so it is available 24/7.

Types: Shared, VPS, Dedicated, Cloud Hosting.

Example: GoDaddy, Hostinger, Bluehost provide hosting space. You upload files via FTP.

Example Explanation: Hosting is like renting a shop space in a mall (server) to keep your goods (website files) so customers (users) can visit anytime.

E) DNS

Explanation: DNS = Domain Name System. Converts domain name to IP address. Like phonebook.

Syntax / Working:

```
User types google.com → DNS → Returns 142.250.195.46 → Browser contacts that IP
```

Example: When you type `abccollege.com`, DNS server resolves it to its IP and then connection happens.

---

7. Client/Server Model

Explanation: Client/Server is distributed architecture where Client (requester, e.g., Browser) requests resources and Server (provider) responds. Web works on this model.

Syntax / Diagram:

```
[CLIENT - Browser/Laptop] --HTTP Request--> [SERVER - Stores Website]
[CLIENT] <--HTTP Response (HTML/CSS/JS)-- [SERVER]

Client = Front-end (What user sees)
Server = Back-end + Database (What user doesn't see)
```

Working Steps:

1. User enters URL in Client
2. DNS resolves Domain → IP
3. Client sends HTTP Request to Server IP
4. Server finds file
5. Server sends HTTP Response
6. Browser renders page

Example: You as client request Instagram profile. Instagram server sends your profile data from database.

Example Explanation: Like customer (client) ordering food via waiter. Kitchen (server) prepares and sends. Client needs server for data.

---

8. Types of Websites

Explanation: Websites classified by functionality and design approach.

A) Based on Functionality:

Type	Explanation	Example
Static	Fixed content, same for all, only HTML/CSS, no DB	Portfolio, Company brochure
Dynamic	Content changes per user, uses DB + Backend	Facebook, Amazon
E-Commerce	Online buying/selling	Flipkart, Amazon
Portal	Aggregates info from many sources	Yahoo, Student portal
CMS/Blog	Content can be managed by admin	WordPress blog
SPA	Single Page Application, loads once, updates dynamically	Gmail, Google Maps

B) Based on Design Approach:

Responsive Website:

- **Explanation:** Layout auto adjusts to screen size (Mobile, Tablet, PC) using fluid grids + media queries.
- **Syntax:** `@media screen and (max-width: 768px) { ... }`
- **Example:** Modern websites like Amazon adjusts in mobile.
- **Example Explanation:** One website works on all devices because CSS changes layout based on screen width.

Website vs Web Application:

- Website = Informative (News site)
- Web Application = Interactive task-based (Google Docs, Canva)

---

9. Front-End, Back-End, Full-Stack

Front-End

Explanation: Client side part user directly sees and interacts with.

Technologies (Syntax):

```
HTML = Structure / Skeleton
CSS = Style / Design
JavaScript = Behavior / Interactivity
```

Frameworks: React.js, Angular, Vue.js, Bootstrap, Tailwind CSS.

Example: Button color, menu layout, image gallery you see.

Example Explanation: Front-end developer converts Figma design to working website using HTML/CSS/JS.

Back-End

Explanation: Server side part user cannot see. Handles database, logic, security.

Syntax / Components:

```
Back-End = Server (Node.js/Apache) + Language (PHP/Python/Java) + Database
(MySQL/MongoDB)
API = Bridge between Front-End and Back-End
```

Example: When you login, backend checks username/password from database, if correct allows entry.

Example Explanation: Front-end is restaurant dining area visible to customer, back-end is kitchen where cooking (logic) and storing ingredients (database) happens.

Full-Stack

Explanation: Developer who can work BOTH front-end and back-end.

Syntax / Popular Stacks:

```
MERN = MongoDB + Express.js + React.js + Node.js (Most Asked)
MEAN = MongoDB + Express + Angular + Node.js
LAMP = Linux + Apache + MySQL + PHP
Full-Stack = Front-End + Back-End + Database + Git
```

Example: A full-stack developer can build complete Amazon clone alone: product page (front-end) + payment & database (back-end).

Comparison Table:

Front-End	Back-End	Full-Stack
Client Side	Server Side	Both
HTML/CSS/JS	PHP/Python/Java/Node	All
Visible	Not visible	Both

---

10. VS Code Editor

Explanation: Visual Studio Code is free, open source, lightweight code editor made by Microsoft. Most popular for web design due to extensions, Emmet built-in, integrated terminal & Git.

Syntax / Interface Structure:

```
VS Code = Activity Bar (left icons) + Side Bar (file explorer) +
          Editor (main coding area) + Panel (terminal bottom) +
          Status Bar (line no, language)
```

Features:

1. Emmet (Built-in):

- **Explanation:** Shortcut to write HTML fast.
- **Syntax:** `! + Tab` = Full HTML boilerplate
- **Example:** Type `ul>li*3` + Tab → Creates ul with 3 li
- **Example Explanation:** Saves 80% typing time.

2. Essential Extensions:

Extension	Explanation	Syntax/Use
Live Server	Live preview without manual refresh	Right click → Open with Live Server
Prettier	Code formatter	Alt+Shift+F formats code
Auto Rename Tag	Auto renames closing tag	Change opening tag, closing auto changes
CSS Peek	Jump from HTML to CSS definition	Ctrl+click on class

3. Shortcuts:

Shortcut	Explanation
`Ctrl + /`	Comment line
`Alt + Up/Down`	Move line up/down
`Ctrl + D`	Select next same word
`Ctrl + ``	Open Terminal
`Ctrl + B`	Toggle Sidebar

Example Program for VS Code Workflow:

```
<!-- In VS Code, type ! + Tab → This boilerplate auto generated -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Document</title>
</head>
<body>
    <h1>Hello from VS Code</h1>
</body>
</html>
```

Example Explanation: Student creates new file ````html

<title>Document</title>

Hello from VS Code

```

**Example Explanation:** Student creates new file `index.html` in VS Code Explorer, types `!` + Tab, gets full structure, writes code, right clicks → Open with Live Server → Browser opens with live reload. This fast workflow is why VS Code is industry standard.

**Alternatives:** Sublime Text, Atom, Brackets, Notepad++.

---

## 11. Web Design Principles & Responsive Design

**Explanation:** Good web design follows principles: Simplicity (less clutter), Consistency (same fonts/colors throughout), Readability (proper contrast & font size), Mobile-First (design for mobile first), Fast Loading (optimize images).

**Responsive Design Syntax:**

```
<!-- Viewport Meta Tag - Mandatory for responsive -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Flexible Image -->
<img src="pic.jpg" style="max-width:100%;">

<!-- Media Query -->
<style>
/* Default for PC */
.box { width: 30%; }
/* For mobile <=768px */
@media screen and (max-width: 768px) {
    .box { width: 100%; }
}
</style>
```

**Example:** On PC 3 boxes in row (30% each), on mobile 1 box per row (100%) via media query.

**Example Explanation:** Responsive makes one website work on mobile, tablet, PC. Without it, mobile users need to scroll horizontally.

**Conclusion:** Web Design Overview covers foundation from Internet infrastructure to practical tool VS Code. Understanding Client/Server working, difference between Internet and WWW, types of websites, and role of frontend/backend/fullstack provides base for learning HTML, CSS, JS.
```

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

2026

Year:

I

Semester:

I

Unit:

I

Topic:

HTML5 Document Structure & Semantic Elements

---

1. What is HTML5?

Explanation: HTML5 is latest version of HyperText Markup Language (2014, W3C). It added semantic tags, native audio/video, new form types, and removed need for Flash. Focus is meaning, not just presentation.

Syntax Difference:

HTML4 (Old)	HTML5 (New)
`<!DOCTYPE html PUBLIC...>` long	`<!DOCTYPE html>` simple
`<div id="header">`	`<header>` semantic
Needs Flash for video	`<video>` native

Example: HTML4 needed `<object>` for video, HTML5 simply `<video src="movie.mp4" controls></video>`

Example Explanation: HTML5 makes code readable and SEO friendly. Search engine understands `<header>` is header, but didn't understand `<div id="header">`.

---

2. HTML5 Basic Document Structure - Skeleton

Explanation: Every HTML5 page must have DOCTYPE, html root, head with meta info, body with visible content.

Syntax Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Title here</title>
</head>
<body>
    <!-- Visible content -->
</body>
</html>
```

Example Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My First HTML5 Page</title>
</head>
<body>
    <h1>Hello World</h1>
    <p>This is first HTML5 document.</p>
</body>
</html>
```

Example Explanation:

- `<!DOCTYPE html>` tells browser HTML5.
- `html lang="en"` root + language helps screen reader pronounce.
- `head` has non-visible info (charset, title).
- `body` has visible h1, p.
- This skeleton is compulsory for valid HTML5.

---

3. Explanation of Individual Structure Tags

A) `<!DOCTYPE html>`

Explanation: Declaration, not tag. Tells browser document is HTML5. Must be first line. Without it browser goes to Quirks Mode and renders inconsistently.

Syntax: `<!DOCTYPE html>` - Case insensitive but uppercase convention.

Example: First line of every HTML file you write in exam must be `<!DOCTYPE html>`.

Example Explanation: If you miss DOCTYPE, browser assumes old HTML and layout may break. So always write.

B) `<html>` **Tag**

Explanation: Root element that contains entire page. lang attribute helps screen readers and SEO.

Syntax: `<html lang="en"> ... </html>`

Example:

```
<html lang="ta">  <!-- Tamil language -->
<html lang="en">  <!-- English -->
```

Example Explanation: lang="en" tells screen reader to pronounce in English accent. If your page is Tamil, use lang="ta" for correct pronunciation.

C) `<head>` **Tag**

Explanation: Contains meta information not visible. Includes charset, viewport, description, title.

Syntax:

```
<head>
    <meta charset="UTF-8">
    <meta name="description" content="...">
    <title>Page Title in Tab</title>
</head>
```

Example:

```
<head>
    <meta charset="UTF-8">
    <title>ABC College - Web Design Notes</title>
</head>
```

Example Explanation: charset UTF-8 supports all languages, title shows in browser tab and Google search results. Without head, SEO poor.

D) `<body>` Tag

Explanation: Contains all visible content: headings, paragraphs, semantic layout (header, nav, main etc).

Syntax: `<body> visible content </body>`

Example: All semantic tags you write go inside body.

---

4. What are Semantic Elements?

Explanation: Semantic = Meaningful. Semantic elements are tags whose name clearly describes their purpose to browser, developer and search engine. Ex: header = header content, footer = footer.

In HTML4 we used `<div id="header">Links</div>` - Browser didn't know it is navigation. In HTML5 `<nav>Links</nav>` - browser instantly knows it is navigation.

Syntax Categories:

Non-Semantic (No Meaning)	Semantic (Meaningful)
`<div>`, `<span>`	`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`

Example:

```
<!-- Non-Semantic HTML4 way -->
<div id="header">ABC College</div>
<div id="nav"><a href="#">Home</a></div>

<!-- Semantic HTML5 way - Same but meaningful -->
<header>ABC College</header>
<nav><a href="#">Home</a></nav>
```

Example Explanation: Both display same visually, but second is SEO friendly because Google understands nav is navigation and header is top introduction. Screen reader can jump directly to nav.

Benefits:

1. SEO - Google ranks higher
2. Accessibility - Screen reader navigation easy
3. Readability - Developer understands quickly

---

5. Detailed Study of Semantic Tags - With Syntax, Example, Explanation

A) `<header>`

Explanation: Represents introductory content - logo, site title, intro. Can be multiple per page (site header + article header).

Syntax: `<header> ... </header>`

Example:

```
<header>
    <h1>ABC College</h1>
    <p>Welcome to Web Design Notes</p>
</header>
```

Example Explanation: This header contains college name and welcome note. It is at top of page. Screen reader identifies it as banner region. Can also be used inside article for article title.

B) `<nav>`

Explanation: Contains major navigation links - main menu.

Syntax: `<nav> <a href="">Link</a> </nav>`

Example:

```
<nav>
    <a href="index.html">Home</a> |
    <a href="about.html">About</a> |
    <a href="contact.html">Contact</a>
</nav>
```

Example Explanation: nav groups navigation links together. Use only for primary navigation, not all links. Helps screen reader user jump directly to navigation.

C) `<main>`

Explanation: Contains dominant unique content of page. Only ONE per page. Must NOT be inside header, footer, article, aside.

Syntax: `<main> ... main content ... </main>`

Example:

```
<main>
    <h2>Main Content Starts Here</h2>
    <p>This page is about Semantic Elements.</p>
</main>
```

Example Explanation: If page repeats header/footer on every page, main is what changes. Screen reader has shortcut "Skip to main content" which jumps to main tag directly for blind users.

D) `<section>`

Explanation: Thematic grouping of content with heading, like chapter in book. Should have h2-h6.

Syntax: `<section><h2>Heading</h2><p>Content</p></section>`

Example:

```
<section>
    <h2>What is Internet?</h2>
    <p>Internet is a global network...</p>
</section>

<section>
    <h2>What is WWW?</h2>
    <p>WWW is a service on Internet...</p>
</section>
```

Example Explanation: Each section is independent topic with its own heading. Use when content can be grouped under one heading. Don't use section as generic wrapper if no heading - use div.

E) `<article>`

Explanation: Independent self-contained composition that can be distributed alone - blog post, news, product card.

Syntax: `<article><h3>Title</h3><p>Content</p></article>`

Example:

```
<article>
    <h2>How to Learn HTML5</h2>
    <p>Article written by John on 16 July 2026...</p>
</article>
```

Example Explanation: This article can be taken alone and shared on another site and still make sense. That is test for article.

Important Difference: Section vs Article:

Section	Article
Part of whole (Chapter in book)	Whole in itself (Book itself)
Groups related content	Independent content
Needs heading	Can stand alone
Example: Latest News section contains many articles	Example: Each news is article

Example - Article inside Section:

```
<section>
    <h2>Latest News</h2>
    <article>
        <h3>News 1: New Chrome Update</h3>
        <p>Chrome released new version...</p>
    </article>
    <article>
        <h3>News 2: HTML6 Rumors</h3>
        <p>Future of HTML...</p>
    </article>
</section>
```

Example Explanation: Section groups all news under heading Latest News, each article is individual news that can stand alone.

F) `<aside>`

Explanation: Sidebar content tangentially related to surrounding content - ads, related links, author info.

Syntax: `<aside> ... sidebar ... </aside>`

Example:

```
<aside>
    <h3>Related Topics</h3>
    <ul>
        <li>HTML Basics</li>
        <li>CSS Overview</li>
    </ul>
</aside>
```

Example Explanation: If main talks about HTML5, aside can list related topics. Inside article, aside can be author bio related to that article.

G) `<footer>`

Explanation: Bottom of page/section - copyright, contact, back to top.

Syntax: `<footer> ... </footer>`

Example:

```
<footer>
    <p>Copyright 2026 ABC College</p>
    <address>
        Contact: webmaster@abccollege.com<br>
        Meenambakkam, Chennai
    </address>
</footer>
```

Example Explanation: Footer typically contains copyright and address. Can be multiple - page footer and article footer.

H) `<figure>` and `<figcaption>`

Explanation: figure is self-contained unit like image/diagram/code. figcaption is caption for it. Must be first or last child of figure.

Syntax: `<figure><img src=""><figcaption>Caption</figcaption></figure>`

Example:

```
<figure>
    <img src="internet.jpg" alt="Diagram of Internet">
    <figcaption>Fig.1 - Structure of Internet</figcaption>
</figure>
```

Example Explanation: figure groups image + caption as single reference unit. Screen reader understands they belong together.

I) Other Semantic Tags - time, mark, address, details

Explanation:

- time = Date/time machine readable
- mark = Highlighted text
- address = Contact info
- details/summary = Collapsible widget

Syntax & Example:

```
<p>Exam Date: <time datetime="2026-09-20">20th Sept 2026</time></p>
<!-- datetime machine format, text human format -->
```

```
<p>Important: <mark>Semantic tags are compulsory in HTML5</mark></p>
<!-- Yellow highlight like highlighter -->

<address>
    ABC College<br>Meenambakkam
</address>

<details>
    <summary>Click to see Advantages</summary>
    <p>1. Better SEO</p>
    <p>2. Better Accessibility</p>
</details>
```

Example Explanation: time datetime helps search engine understand date, mark highlights important for reference, address gives contact info semantics, details creates accordion without JS - summary is visible heading.

---

6. Standard Page Layout Structure

Explanation: Typical semantic layout combines all tags in order.

Syntax / Diagram:

```
<header> Logo + Title
<nav> Home | About | Contact
<main>
   <section> (group)
     <article> Article 1
     <article> Article 2
   <aside> Sidebar
<footer> Copyright
```

Example - Full Page:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Semantic Layout Example</title>
</head>
<body>

    <header>
        <h1>My Tech Blog</h1>
        <p>Learn Web Design Easily</p>
    </header>
```

```
    <nav>
        <a href="#home">Home</a>
        <a href="#html">HTML5</a>
    </nav>

    <main>
        <section>
            <h2>Latest Articles</h2>
            <article>
                <h3>What is Semantic HTML?</h3>
                <p>Published on <time datetime="2026-07-16">16 July 2026</time>
</p>
                <p>Semantic HTML uses meaningful tags.</p>
                <figure>
                    <img src="semantic.jpg" alt="Semantic Tags">
                    <figcaption>Semantic Tags Overview</figcaption>
                </figure>
            </article>
            <article>
                <h3>Why Use Main Tag?</h3>
                <p>Main contains dominant content.</p>
                <p><mark>Only one main tag per page</mark></p>
            </article>
        </section>
    </main>

    <aside>
        <h3>About Author</h3>
        <p>Web Design Faculty</p>
    </aside>

    <footer>
        <p>&copy; 2026 My Tech Blog</p>
        <address>Email: blog@example.com</address>
    </footer>

</body>
</html>
```

Example Explanation: This code demonstrates exam pattern: DOCTYPE first, html lang, head meta, body contains header → nav → main (section → articles with time, figure, mark) → aside → footer with address. It uses 10 semantic tags, ideal for 10 marks. main appears only once. Each section/article has heading. This layout gets full marks.

---

7. Best Practices - DOs and DON'Ts

DOs - With Reasoning:

- **Explanation:** Use main only once because it is dominant unique content. Syntax: one `<main>` per page.
- **Explanation:** Every section/article must have heading h2-h6 because outlining algorithm needs heading.
- **Explanation:** Use nav only for major navigation, not all links.

DON'Ts:

- Don't put main inside header/footer/article/aside - Semantic rule violation.
- Don't use section as generic wrapper without heading - Use div instead.
- Don't forget lang attribute - Accessibility fails.

Final Trick for Exam: Remember layout H N M A F = **H**eader, **N**av, **M**ain (Section > Article), **A**side, **F**ooter. Always start with `<!DOCTYPE html>` and use at least 5 semantic tags for full marks.

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

2026

Year:

I

Semester:

I

Unit:

I

Topic:

Text, Hyperlinks, Navigation - Images and Media

---

1. Text Elements Overview

Explanation: Text elements are core of HTML. They are divided into Block-level (starts new line, full width) like h1-p-div and Inline-level (same line) like a-b-img-span. Proper use helps SEO and screen readers.

---

2. Headings h1-h6

Explanation: Headings define hierarchy - h1 most important big, h6 least small. Use one h1 per page ideally, and follow order h1→h2→h3 without skipping for accessibility.

Syntax: `<h1>Heading</h1>` to `<h6>Smallest Heading</h6>`

Example:

```
<h1>Main Title - ABC College</h1>
<h2>Section Title - Courses</h2>
```

```
<h3>Sub Section - BCA</h3>
```

Example Explanation: Search engines give more weight to h1, screen readers navigate via headings list. So h1 for site title, h2 for sections, h3 for subsections.

---

3. Paragraphs, Line Break, Horizontal Rule

Explanation: p defines paragraph with auto space before/after. br is empty tag for line break within paragraph. hr is thematic break/horizontal line.

Syntax:

```
<p>Paragraph text</p>
<p>Line1<br>Line2</p>
<hr>
```

Example:

```
<p>This is first paragraph.</p>
<p>First line<br>Second line after break</p>
<hr>
<p>After horizontal line</p>
```

Example Explanation: p creates new paragraph block, br forces second line same paragraph (like address), hr separates sections visually.

---

4. Text Formatting - Semantic vs Visual

Explanation: HTML has visual formatting (only look) vs semantic formatting (meaning + accessibility).

Visual (Only Look)	Semantic (Meaning + SEO)
<b> bold look	important bold
<i> italic look	stressed emphasis
<u> underline	<mark> highlighted

Syntax & Examples:

```
<p><b>Bold</b> - Visual only (Non-semantic)</p>
<p><strong>Strong</strong> - Important (Semantic, screen reader stresses)</p>

<p><i>Italic</i> - Visual only</p>
<p><em>Emphasized</em> - Semantic stress</p>
```

```
<p><mark>Highlighted Text</mark></p>
<p><small>Small Text - Copyrights</small></p>
<p><del>Rs 1000</del> <ins>Now Rs 800</ins></p>
<p>H<sub>2</sub>O and a<sup>2</sup></p>
```

Example Explanation: `<b>` and `<strong>` look both bold, but screen reader reads strong with importance, and Google gives more weight to strong. Similarly i vs em. Use mark for highlighter effect, del for deleted with strikethrough, ins for inserted with underline, sub for subscript like water, sup for superscript like square.

---

5. Other Semantic Text Tags

Explanation: Tags for specific purposes: abbr for abbreviation, blockquote for long quote, q for short inline quote, cite for book/movie title, code for programming code, pre for preserving spaces+lines, kbd for keyboard key.

Syntax & Examples:

```
<p>This is <abbr title="HyperText Markup Language">HTML</abbr></p>
<!-- title shows full form on hover -->

<blockquote cite="https://source.com">
    Long quotation from another source - Block level indented.
</blockquote>

<p>He said <q>Short inline quote</q> in class.</p>

<p><cite>HTML and CSS Book</cite> by Jon Duckett</p>

<p>Code: <code>console.log('hello')</code></p>

<pre>
Pre preserves
  spaces and
    line breaks as typed
</pre>

<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save</p>
```

Example Explanation: abbr helps users know full form via tooltip, blockquote for long quotes, q adds quotes automatically via browser, cite italics by default for title, code monospace font for code, pre keeps formatting (useful for poetry/code), kbd looks like keyboard key.

---

6. Hyperlinks - a Tag

Explanation: Hyperlink is clickable reference to another document/section. Created with anchor `<a>` tag. It forms web of interlinked documents.

Syntax: `<a href="URL" target="" title="" download>Link Text</a>`

Important Attributes:

Attribute	Explanation	Example
href	Destination URL - Mandatory	href="about.html"
target	Where to open: _self same tab (default), _blank new tab	target="_blank"
title	Tooltip on hover	title="Go to About"
download	Forces download	download
rel	Relationship, security	rel="noopener noreferrer"

Example:

```
<a href="https://www.google.com">Go to Google</a>
<a href="about.html" title="About Page">About Us</a>
<a href="https://wikipedia.org" target="_blank" rel="noopener noreferrer">Open
Wikipedia in New Tab</a>
<a href="notes.pdf" download>Download Notes</a>
```

Example Explanation: First link absolute external, second relative internal with tooltip, third opens in new tab with security rel (prevents new page accessing old page via window.opener), fourth downloads file instead of navigating.

---

7. Types of Links

Explanation: Links classified by destination type.

Syntax & Examples:

```
<!-- 1. Absolute URL - Full address external -->
<a href="https://www.flipkart.com">Flipkart - External</a>

<!-- 2. Relative URL - Inside same website -->
<a href="contact.html">Contact - Same folder</a>
<a href="images/photo.jpg">View Photo - Subfolder</a>

<!-- 3. Email Link - mailto: -->
<a href="mailto:student@college.com">Send Email</a>
<a href="mailto:info@example.com?subject=Exam Doubt">Mail with Subject</a>

<!-- 4. Phone Link - tel: -->
<a href="tel:+919876543210">Call Us</a>

<!-- 5. Image as Link - Image inside anchor -->
<a href="https://www.google.com">
```

```
    <img src="google-logo.png" alt="Click to go to Google">
</a>
```

Example Explanation: Absolute for external sites needs full URL, relative for internal so site portable even if domain changes, mailto opens email client with to address, tel on mobile triggers call, image as link makes whole image clickable.

---

8. Internal Navigation - Bookmarks

Explanation: Bookmark links jump to specific part of same page without reload. Uses id attribute as destination and #id as link. Crucial for single-page websites and Back to Top.

Syntax:

```
<!-- Destination with id -->
<h2 id="section1">Section 1</h2>

<!-- Link to destination -->
<a href="#section1">Go to Section 1</a>

<!-- Back to Top -->
<h2 id="top">Top</h2>
...
<a href="#top">Back to Top</a>
```

Example Program:

```
<h2 id="top">Table of Contents</h2>
<ul>
    <li><a href="#chapter1">Go to Chapter 1</a></li>
    <li><a href="#chapter2">Go to Chapter 2</a></li>
</ul>
<hr>
<h2 id="chapter1">Chapter 1: Internet</h2>
<p>Content...</p>
<a href="#top">Back to Top</a>
<hr>
<h2 id="chapter2">Chapter 2: WWW</h2>
<p>Content...</p>
<a href="#top">Back to Top</a>
```

Example Explanation: First list links to sections same page. Clicking #chapter1 scrolls to element with id chapter1. Back to Top links at bottom bring user back to top element with id top. This avoids manual scrolling.

---

9. Navigation Menu with nav

Explanation: Navigation menu is structured set of links to navigate site. HTML5 uses semantic nav + ul + li + a for best accessibility and SEO.

Syntax:

```
<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
    </ul>
</nav>
```

Example Program:

```
<header>
    <h1>ABC College</h1>
    <nav>
        <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="about.html">About College</a></li>
            <li><a href="courses.html">Courses</a></li>
            <li><a href="contact.html">Contact</a></li>
        </ul>
    </nav>
</header>
```

Example Explanation: nav indicates navigation landmark for screen reader, ul removes need for divs, li each menu item, a actual link. Types: Primary navigation in header, Secondary in aside, Footer navigation (privacy, terms), Breadcrumb (Home > Courses > Web Design).

---

10. Images - img Tag

Explanation: img embeds images. It is empty tag (no closing). Two mandatory attributes: src (source path) and alt (alternative text). Alt is compulsory for accessibility (screen reader) and SEO and shows when image fails.

Syntax: `<img src="path" alt="description" width="" height="" title="">`

Attributes:

- src = Relative (logo.jpg) or Absolute (https://.../banner.jpg)
- alt = Meaningful description, empty alt="" for decorative
- width/height = Reserve space, prevent layout shift
- title = Tooltip

Example Program:

```
<!-- Relative path -->
<img src="logo.jpg" alt="College Logo" width="200" height="100" title="ABC
College Logo">

<!-- Absolute URL -->
<img src="https://www.example.com/banner.jpg" alt="College Banner Photo">

<!-- Image as Link -->
<a href="https://www.google.com">
    <img src="google-logo.png" alt="Click to go to Google">
</a>
```

Example Explanation: First image from same folder with alt College Logo, width height reserve space, title shows tooltip. Second from internet absolute URL. Third image clickable as link to Google. Alt text must be meaningful, not "image1".

---

11. Figure and Figcaption

Explanation: When image needs caption or is referenced as single unit, use figure + figcaption semantic grouping. figcaption must be first or last child of figure.

Syntax:

```
<figure>
    <img src="" alt="">
    <figcaption>Caption text</figcaption>
</figure>
```

Example:

```
<figure>
    <img src="lab.jpg" alt="Computer Lab with 30 systems" width="300">
    <figcaption>Fig 1: Computer Lab - CSE Department</figcaption>
</figure>
```

Example Explanation: figure groups image + caption together so screen reader knows they belong together. Use figure for important images with caption, use only img for decorative or inline images.

---

12. Image Maps

Explanation: Makes different parts of single image clickable to different links. Uses img usemap attribute referencing map name, map element containing area elements.

Syntax:

```
<img src="image.jpg" usemap="#mapname" alt="">
<map name="mapname">
    <area shape="rect" coords="x1,y1,x2,y2" href="" alt="">
    <area shape="circle" coords="x,y,radius" href="">
</map>
```

Example Program:

```
<img src="shapes.jpg" alt="Shapes" usemap="#shapemap" width="400">

<map name="shapemap">
    <area shape="rect" coords="0,0,100,100" href="rectangle.html"
alt="Rectangle Area">
    <area shape="circle" coords="200,50,50" href="circle.html" alt="Circle
Area">
</map>
```

Example Explanation: shapes.jpg is single image with rectangle and circle drawn. First area rect coords 0,0,100,100 is top-left 100x100 rectangle clickable to rectangle.html. Second circle centered at 200,50 radius 50 clickable to circle.html. Useful theory but modern CSS replaced it.

---

13. Audio Tag

Explanation: HTML5 native audio without Flash plugin. Needs src, controls for play/pause/volume UI.

Syntax:

```
<audio src="file.mp3" controls autoplay loop muted></audio>

<!-- Multiple sources for browser compatibility -->
<audio controls>
    <source src="file.ogg" type="audio/ogg">
    <source src="file.mp3" type="audio/mpeg">
    Browser doesn't support audio.
</audio>
```

Attributes: controls (show controls), autoplay (auto start - avoid bad UX), loop (repeat), muted (start muted).

Example Program:

```
<h2>Simple Audio</h2>
<audio src="anthem.mp3" controls>
    Your browser does not support audio tag.
</audio>
```

```
<h2>With multiple formats</h2>
<audio controls>
    <source src="anthem.ogg" type="audio/ogg">
    <source src="anthem.mp3" type="audio/mpeg">
    Your browser does not support audio element.
</audio>
```

Example Explanation: First simple audio with src directly. Second with two sources - browser tries.ogg first if supports, else mp3. Text between tags shows if both fail. controls mandatory else no UI.

---

14. Video Tag

Explanation: Native video embedding. Supports width, height, poster (thumbnail before play), controls, autoplay, loop, muted. Also needs multiple formats mp4/webm/ogv for cross browser. track element for subtitles.

Syntax:

```
<video src="video.mp4" controls width="400" poster="thumb.jpg"></video>

<video controls width="500" poster="poster.jpg">
    <source src="video.mp4" type="video/mp4">
    <source src="video.webm" type="video/webm">
    <track src="subs_en.vtt" kind="subtitles" srclang="en" label="English">
    Sorry no support.
</video>
```

Example Program:

```
<video src="lecture.mp4" controls width="400" height="250"
poster="thumbnail.jpg">
    Your browser does not support video tag.
</video>

<video controls width="500" poster="lecture_poster.jpg">
    <source src="lecture.mp4" type="video/mp4">
    <source src="lecture.webm" type="video/webm">
    <track src="subtitles_en.vtt" kind="subtitles" srclang="en"
label="English">
    <track src="subtitles_ta.vtt" kind="subtitles" srclang="ta" label="Tamil">
    Your browser does not support HTML5 Video.
</video>
```

Example Explanation: First simple video with poster thumbnail. Second with multiple sources + subtitles: mp4 most supported, webm for Firefox, track vtt file for subtitles English/Tamil for deaf users and SEO indexing. poster image shows before play.

---

15. Iframe - Embedding External Content

Explanation: iframe embeds another webpage inside current page. Used for YouTube videos, Google Maps, external ads.

Syntax:

```
<iframe src="URL" width="" height="" title="" frameborder="0" allowfullscreen>
</iframe>
```

Attributes: src, width, height, title (mandatory for accessibility screen reader announces), frameborder border, allowfullscreen.

Example Program:

```
<h2>YouTube Video via Iframe</h2>
<iframe width="560" height="315"
src="https://www.youtube.com/embed/dQw4w9WgXcQ" title="YouTube Video"
frameborder="0" allowfullscreen>
</iframe>

<h2>Google Map</h2>
<iframe src="https://www.google.com/maps/embed?pb=..." width="400" height="300"
title="College Location Map"></iframe>

<h2>Embed another HTML page</h2>
<iframe src="about.html" width="500" height="300" title="About Page Content">
</iframe>
```

Example Explanation: First iframe embeds YouTube - Note embed URL format /embed/ not /watch?v=. title YouTube Video for accessibility. allowfullscreen allows fullscreen button. Second embeds Google Maps location. Third embeds own about.html page inside current page. Without title, screen reader cannot identify iframe purpose, so title compulsory.

---

16. Complete Combined Example

Explanation: Single page combining all topics is asked for 10 marks - Text formatting, Hyperlinks, Navigation, Images, Media.

Syntax: Header + Nav bookmark links + Sections with text formatting + Figure + Audio + Video + Iframe + Footer with mailto/tel.

Example Program - Full Portfolio:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>My Portfolio</title>
</head>
<body>

    <header>
        <h1>John Doe - Web Designer</h1>
        <nav>
            <ul>
                <li><a href="#about">About</a></li>
                <li><a href="#gallery">Gallery</a></li>
                <li><a href="#media">My Work</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <section id="about">
            <h2>About Me</h2>
            <p>I am <strong>Computer Science</strong> student at <abbr
title="ABC College">ABCCE</abbr>. My goal is <mark>Full Stack Developer</mark>.
</p>
            <p>Formula: H<sub>2</sub>O and a<sup>2</sup>. Use <code>&lt;a&gt;
</code> for link.</p>
            <blockquote>"Design is how it works" - Steve Jobs</blockquote>
        </section>

        <section id="gallery">
            <h2>Gallery</h2>
            <figure>
                <img src="project1.jpg" alt="My First Project" width="300">
                <figcaption>Fig 1: First HTML Project</figcaption>
            </figure>
        </section>

        <section id="media">
            <h2>Media</h2>
            <h3>Audio</h3>
            <audio controls><source src="intro.mp3" type="audio/mpeg"></audio>
            <h3>Video</h3>
            <video controls width="400" poster="poster.jpg"><source
src="demo.mp4" type="video/mp4"></video>
            <h3>Map</h3>
            <iframe src="https://www.google.com/maps/embed..." width="400"
height="250" title="Map"></iframe>
        </section>
    </main>

    <footer id="contact">
        <h2>Contact</h2>
        <address>
            Email: <a href="mailto:john@example.com">john@example.com</a><br>
            Phone: <a href="tel:+919876543210">+91 9876543210</a>
```

```
        </address>
        <p><a href="#top">Back to Top</a></p>
    </footer>

</body>
</html>
```

Example Explanation: This single program demonstrates everything: h1-h2 headings, strong/mark/sub/sup/code/abbr/blockquote formatting, nav with bookmark links #about #gallery, figure with figcaption, audio/video with controls+poster, iframe map with title, footer address with mailto/tel email/phone links. Ideal to write for any 10-mark question asking to design webpage with text, links, navigation, images, media.

Best Practices Recap:

- Every img must have alt meaningful.
- Every iframe must have title.
- Use strong not b for importance, em not i for emphasis.
- For external links use target="_blank" + rel="noopener noreferrer".
- For bookmark, id and #id must match exact case.
- Audio/video always give controls, multiple sources for compatibility.
- Avoid autoplay with sound - bad UX.

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

> **2026**

Year:

> **I**

Semester:

> **I**

Unit:

> **I**

Topic:

> **Forms and Validation - Tables**

PART A: FORMS

1. What is Form?

Explanation: Form is a section of document that collects user data and sends to server for processing. Used for Login, Registration, Search, Feedback. Works on Client/Server model: User fills in browser (client), on submit data goes to server (action), server processes and responds.

Syntax Basic Skeleton:

```
<form action="URL" method="get/post">
    <!-- input fields -->
    <input type="submit" value="Submit">
</form>
```

Example:

```
<form action="login.php" method="post">
    <input type="text" name="username" placeholder="Username">
    <input type="password" name="password" placeholder="Password">
    <input type="submit" value="Login">
</form>
```

Example Explanation: form container with action="login.php" means data sent to login.php, method post hides data in body. Inside two text fields with name attributes (without name data won't go) and submit button. On clicking Login, browser sends username and password to server.

---

2. Form Tag Attributes

Explanation: Attributes control where and how form data is sent.

Syntax & Attributes Table:

Attribute	Explanation	Syntax Example
action	URL where data sent	`action="submit.php"`
method	GET or POST how data sent	`method="post"`
enctype	Encoding type - Must be multipart/form-data for file upload	`enctype="multipart/form-data"`
autocomplete	Browser autofill on/off	`autocomplete="off"`
novalidate	Disable HTML5 validation	`novalidate`
target	Where to show response _self/_blank	`target="_blank"`

Example:

```
<form action="process.php" method="post" autocomplete="on"
enctype="multipart/form-data"></form>
<form action="test.php" novalidate></form>
```

Example Explanation: First form for file upload needs post + multipart/form-data + autocomplete on allows browser to remember. Second with novalidate disables browser validation for custom JS validation testing.

---

3. GET vs POST

Explanation: GET appends data to URL visible, limited, bookmarkable, used for search. POST hides data in body, unlimited, more secure, used for login/file upload.

Syntax Difference:

```
<!-- GET - Visible -->
<form method="get" action="search.php">
    <input type="search" name="q">
</form>
<!-- URL becomes search.php?q=html -->

<!-- POST - Hidden -->
<form method="post" action="login.php">
    <input type="password" name="pwd">
</form>
```

Comparison Table:

Point	GET	POST
Visible in URL?	YES `?name=john`	NO Hidden
Length Limit	~2048 chars limited	No limit
Security	Less secure, not for password	More secure
Bookmark	Can bookmark	Cannot
Use	Searching, Filtering	Login, Registration, File Upload

Example Explanation: Search form uses GET so you can share URL `search?q=html` with friend and same results. Login must use POST because password should not show in URL.

---

4. Input Tag - Heart of Forms

Explanation: Input creates different controls based on type attribute. Empty tag. Crucial attributes: name (key sent to server, if no name data not sent), id (unique identifier for label linking and JS), value (default value), placeholder (hint when empty).

Syntax: `<input type="typeName" name="uniqueName" id="id" value="" placeholder="">`

Example:

```
<input type="text" name="fullname" id="fullname" value="" placeholder="Enter
Full Name">
```

Example Explanation: type text single line, name fullname is key sent as fullname=John, id same for label linking, placeholder hint grey text disappears on typing. If you miss name, server won't receive this field's data.

Label Association:

Explanation: Label improves accessibility, clicking label focuses input, screen reader reads label.

Syntax:

```
<!-- Method 1 Best: for/id linking -->
<label for="email">Email:</label>
<input type="email" id="email" name="email">

<!-- Method 2 Wrapping -->
<label>Password: <input type="password" name="password"></label>
```

Example Explanation: For/id method: label for="email" must match input id="email". When user clicks Email text, cursor focuses email field. Essential for exam marks.

---

5. All Input Types - Detailed with Syntax & Example

A) Text Data Inputs

Explanation: For text, email, phone etc. Each type gives built-in validation and mobile keyboard.

Syntax & Examples:

```
<!-- text -->
<input type="text" name="username" placeholder="Username" maxlength="20">
<!-- Explanation: Single line text, maxlength 20 chars max -->

<!-- password -->
<input type="password" name="pwd" placeholder="Password">
<!-- Explanation: Shows dots, secure -->

<!-- email - Auto validates email -->
<input type="email" name="email" placeholder="abc@example.com">
<!-- Explanation: Must contain @ and dot, else browser shows error -->

<!-- tel -->
<input type="tel" name="phone" placeholder="9876543210">

<!-- url -->
<input type="url" name="website" placeholder="https://example.com">
<!-- Explanation: Must be valid http/https URL -->

<!-- search -->
<input type="search" name="search" placeholder="Search here">
<!-- Explanation: For search box, shows clear cross in some browsers -->
```

B) Number and Range

Explanation: For numeric data with min max control.

Syntax & Examples:

```
<input type="number" name="age" min="18" max="100" step="1" value="18">
<!-- Explanation: Only numbers, min 18 max 100, step 1 increment, default 18 --
>

<input type="range" name="volume" min="0" max="100" value="50">
<!-- Explanation: Slider 0-100 default 50 -->

<input type="color" name="favcolor" value="#ff0000">
<!-- Explanation: Color picker returns hex #ff0000 -->
```

C) Date and Time (HTML5 New)

Explanation: Show native calendar/clock picker.

Syntax:

```
<input type="date" name="dob"> <!-- yyyy-mm-dd -->
<input type="time" name="meeting_time">
<input type="month" name="exp_month">
<input type="week" name="week">
<input type="datetime-local" name="appointment">
```

Example Explanation: date shows date picker calendar, time shows time spinner, month shows month+year, datetime-local shows both date and time without timezone.

D) Choice - Radio, Checkbox, File

Explanation: Radio = Choose only ONE from group (same name groups them). Checkbox = Choose MULTIPLE. File = Upload files.

Syntax & Examples:

```
<!-- Radio - Same name = One group -->
<p>Gender:</p>
<input type="radio" id="male" name="gender" value="male" checked>
<label for="male">Male</label>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label>
<!-- Explanation: Both have name gender so only one can be selected. value sent
to server. checked default tick. -->

<!-- Checkbox - Multiple -->
<p>Skills:</p>
<input type="checkbox" id="html" name="skills" value="html" checked>
<label for="html">HTML</label>
<input type="checkbox" id="css" name="skills" value="css">
<label for="css">CSS</label>
```

```
<!-- Explanation: Can tick both HTML and CSS, each with same name but different
value -->

<!-- File -->
<input type="file" name="resume" accept=".pdf,.doc">
<!-- accept filters file types -->
<input type="file" name="photos" multiple accept="image/*">
<!-- multiple allows many files, accept image/* only images -->
```

Important Note for File: Form must have `method="post"` AND `enctype="multipart/form-data"` else file won't upload.

E) Buttons

Explanation: Buttons to submit, reset, or generic.

Syntax:

```
<input type="submit" value="Register"> <!-- Sends form -->
<input type="reset" value="Clear Form"> <!-- Clears all fields -->
<input type="button" value="Click Me"> <!-- Does nothing, needs JS -->
<input type="hidden" name="userid" value="12345"> <!-- Hidden data sent but not
visible -->
<input type="image" src="submit.png" alt="Submit" width="100"> <!-- Image as
submit -->
```

---

6. Other Form Controls (Not input types)

A) Textarea

Explanation: For multiline text like address, feedback. Not empty tag, has closing.

Syntax:

```
<textarea name="address" rows="4" cols="40" placeholder="Enter address">
</textarea>
```

Attributes: rows height lines, cols width chars, maxlength, required, readonly, placeholder.

Example:

```
<label for="address">Address:</label>
<textarea id="address" name="address" rows="4" cols="40" placeholder="Enter
full address" maxlength="200" required></textarea>
```

Example Explanation: 4 rows high 40 cols wide, placeholder hint, maxlength 200 chars limit, required makes it mandatory.

B) Select, Option, Optgroup (Dropdown)

Explanation: Dropdown list. option inside select is each choice. optgroup groups options under label.

Syntax:

```
<select name="course" id="course">
    <option value="">-- Select One --</option>
    <option value="bca">BCA</option>
    <option value="bsc" selected>BSc CS</option>
</select>
<!-- selected = default, value sent to server -->
```

Example with Optgroup:

```
<label for="city">City:</label>
<select id="city" name="city">
    <optgroup label="Tamil Nadu">
        <option value="chennai">Chennai</option>
        <option value="coimbatore">Coimbatore</option>
    </optgroup>
    <optgroup label="Karnataka">
        <option value="bangalore">Bangalore</option>
    </optgroup>
</select>

<!-- Multiple Select -->
<select name="languages" multiple size="4">
    <option value="tamil">Tamil</option>
    <option value="english" selected>English</option>
</select>
```

Example Explanation: First select simple courses, second groups cities by state labels Tamil Nadu/Karnataka, multiple attribute allows selecting many with Ctrl key, size 4 shows 4 options visible, selected pre-selects.

C) Datalist (Autocomplete Suggestions - HTML5)

Explanation: Combines text input + dropdown. User can type custom or choose from suggestions. Different from select where only list choices allowed.

Syntax: input list="id" must match datalist id="id"

```
<label for="browser">Browser:</label>
<input list="browsers" id="browser" name="browser" placeholder="Type or
```

```
Select">
<datalist id="browsers">
    <option value="Chrome">
    <option value="Firefox">
    <option value="Edge">
</datalist>
```

Example Explanation: Input shows list attribute pointing to datalist id browsers. Datalist contains suggestions Chrome/Firefox/Edge. User can type Safari custom even if not in list, unlike select.

D) Fieldset and Legend (Grouping)

Explanation: Fieldset groups related fields into bordered box, legend gives title to that box. Improves semantics and accessibility - screen reader reads legend for each group.

Syntax:

```
<fieldset>
    <legend>Personal Information</legend>
    <label>Name: <input type="text" name="name"></label>
</fieldset>
```

Example:

```
<fieldset>
    <legend>Personal Information</legend>
    <label for="fname">First Name:</label>
    <input type="text" id="fname" name="fname"><br><br>
    <label for="lname">Last Name:</label>
    <input type="text" id="lname" name="lname">
</fieldset>
```

Example Explanation: Fieldset creates box border around two fields, legend Personal Information shows as box title. Good for grouping.

E) Button Tag (More Powerful than input button)

Explanation: Button tag can contain HTML inside (image, formatted text) unlike input button which only plain text.

Syntax:

```
<button type="submit">Submit Form</button>
<button type="reset">Reset</button>
```

```
<button type="button">Normal Button</button>
<button type="submit"><img src="send.png" width="20"> Send</button>
```

F) Output Tag

Explanation: Shows result of calculation.

Syntax:

```
<form oninput="result.value=parseInt(a.value)+parseInt(b.value)">
    <input type="number" id="a" value="10"> +
    <input type="number" id="b" value="20"> =
    <output name="result" for="a b">30</output>
</form>
```

Example Explanation: As user changes a or b numbers, oninput calculates a+b and shows in output 30.

---

7. Input Attributes for Validation

Explanation: Attributes control validation.

Table:

Attribute	Explanation	Syntax Example
required	Must fill	`<input required>`
placeholder	Hint	`placeholder="Enter name"`
readonly	View but not edit, data sent	`readonly` value India readonly
disabled	Can't edit, data NOT sent, greyed	`disabled`
maxlength/minlength	Max/min chars	`maxlength="10" minlength="3"`
min/max	Min/max for number/date	`min="18" max="100"`
pattern	Regex custom validation	`pattern="[0-9]{10}"`
title	Hint for pattern error	`title="10 digit number"`

Example readonly vs disabled:

```
<input type="text" name="country" value="India" readonly> <!-- Data WILL go to
server -->
<input type="text" name="country2" value="India" disabled> <!-- Data WON'T go,
greyed -->
```

Example Explanation: readonly user sees India but can't change, data sent. disabled also can't change but data NOT sent and looks grey disabled.

---

8. HTML5 Form Validation - In Depth

Explanation: Validation checks user input before sending to server. Types: Client Side (HTML5 + JS in browser, fast but can be bypassed) and Server Side (PHP/Java on server, secure final check).

HTML5 provides built-in client side validation: type attribute checks format (email needs @, url needs http), required checks empty, min/max checks range, pattern checks regex.

Syntax & Example Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Validation Demo</title>
</head>
<body>
    <h2>Form with HTML5 Validation</h2>
    <form action="submit.php" method="post">
        <label>Username (3-10 chars, required):</label>
        <input type="text" name="uname" required minlength="3" maxlength="10"
placeholder="Min 3 chars"><br><br>

        <label>Email (Required, auto email check):</label>
        <input type="email" name="email" required
placeholder="abc@example.com"><br><br>

        <label>Age (18-60 only):</label>
        <input type="number" name="age" required min="18" max="60"><br><br>

        <input type="submit" value="Check Validation">
    </form>
</body>
</html>
```

Example Explanation: If you submit empty, browser shows error for required fields automatically no JS needed. type email checks @ symbol, type number with min 18 max 60 checks range 18-60 else error. This is HTML5 built-in validation.

---

9. Pattern Attribute - Regex Validation

Explanation: Pattern allows custom validation using Regular Expression. Most powerful. Must use title to show hint.

Syntax: `pattern="regex" title="hint for user if fails"`

Common Patterns for Exam:

Requirement	Pattern	Title
10 digit number	`[0-9]{10}`	10 digits only
Indian Mobile 6-9 start	`[6-9][0-9]{9}`	10 digit starting 6-9
Only Letters	`[A-Za-z]+`	Only alphabets
6 digit Pincode	`[0-9]{6}`	6 digit pincode
Roll No ABC1234	`[A-Z]{3}[0-9]{4}`	3 uppercase + 4 digits

Example Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Pattern Demo</title>
</head>
<body>
    <h2>Registration with Pattern</h2>
    <form action="register.php" method="post">
        <label>Mobile (10 digits):</label>
        <input type="tel" name="mobile" pattern="[0-9]{10}" required
title="Enter 10 digit number only"><br><br>

        <label>City (Only Alphabets):</label>
        <input type="text" name="city" pattern="[A-Za-z ]+" required
title="Only alphabets allowed"><br><br>

        <label>Pincode:</label>
        <input type="text" name="pin" pattern="[0-9]{6}" maxlength="6" required
title="6 digit pincode"><br><br>

        <label>Roll No (ABC1234):</label>
        <input type="text" name="roll" pattern="[A-Z]{3}[0-9]{4}" required
title="Ex: CSE2025 - 3 uppercase + 4 digits"><br><br>

        <input type="submit" value="Validate">
    </form>
</body>
</html>
```

Example Explanation: Mobile field pattern [0-9]{10} allows exactly 10 digits 0-9, if user types letters or 9 digits, browser shows error with title "Enter 10 digit number only". City pattern [A-Za-z]+ allows only letters and space, no numbers. Pincode 6 digits only. Roll No 3 capital letters + 4 numbers.

---

10. Full Example Programs - Forms

A) Login Form (Most Asked)

Explanation: Simple login with username password required, autofocus on username, remember checkbox.

Syntax & Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Login Form</title>
</head>
<body>
    <h2>Student Login</h2>
    <form action="login.php" method="post" autocomplete="on">
        <fieldset>
            <legend>Login Details</legend>
            <label for="username">Username:</label>
            <input type="text" id="username" name="username" required
placeholder="Enter Username" autofocus><br><br>

            <label for="pwd">Password:</label>
            <input type="password" id="pwd" name="password" required
minlength="6" placeholder="Min 6 characters"><br><br>

            <input type="checkbox" id="remember" name="remember" checked>
            <label for="remember">Remember Me</label><br><br>

            <input type="submit" value="Login">
            <input type="reset" value="Clear">
        </fieldset>
    </form>
</body>
</html>
```

Example Explanation: Fieldset groups login with legend Login Details, username required with autofocus (cursor automatically there on page load), password minlength 6, remember Me checkbox checked by default, submit sends data post to login.php, reset clears.

B) Complete Registration Form (10 Marks)

Explanation: Full admission form covering all input types, fieldset grouping, datalist, file upload, validation.

Syntax & Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Admission Form</title>
```

```
</head>
<body>
    <h1>College Admission Form 2026</h1>
    <form action="admission.php" method="post" enctype="multipart/form-data">

        <fieldset>
            <legend>Personal Details</legend>
            <label for="fullname">Full Name:</label>
            <input type="text" id="fullname" name="fullname" required
minlength="3" maxlength="30" placeholder="Full Name"><br><br>

            <label for="email4">Email:</label>
            <input type="email" id="email4" name="email4" required
placeholder="abc@example.com"><br><br>

            <label for="phone4">Mobile:</label>
            <input type="tel" id="phone4" name="phone4" required pattern="[6-9]
[0-9]{9}" title="10 digit starting 6-9"><br><br>

            <label>Gender:</label>
            <input type="radio" id="m" name="gender" value="male" required>
            <label for="m">Male</label>
            <input type="radio" id="f" name="gender" value="female">
            <label for="f">Female</label><br><br>

            <label for="dob">DOB:</label>
            <input type="date" id="dob" name="dob" required><br><br>

            <label for="color">Favorite Color:</label>
            <input type="color" id="color" name="color" value="#0000ff"><br>
<br>
        </fieldset>

        <fieldset>
            <legend>Academic Details</legend>
            <label for="course2">Course:</label>
            <select id="course2" name="course2" required>
                <option value="">--Select Course--</option>
                <option value="bca">BCA</option>
                <option value="bsc">BSc CS</option>
            </select><br><br>

            <label for="state">State:</label>
            <input list="states" id="state" name="state" placeholder="Select or
Type State">
            <datalist id="states">
                <option value="Tamil Nadu">
                <option value="Karnataka">
            </datalist><br><br>

            <label>Languages:</label>
            <input type="checkbox" id="eng" name="lang" value="english"
checked>
            <label for="eng">English</label>
```

```
            <input type="checkbox" id="tam" name="lang" value="tamil">
            <label for="tam">Tamil</label><br><br>

            <label for="address2">Address:</label><br>
            <textarea id="address2" name="address2" rows="4" cols="40"
required></textarea><br><br>

            <label for="photo">Upload Photo:</label>
            <input type="file" id="photo" name="photo" accept="image/*"
required><br><br>
        </fieldset>

        <input type="checkbox" id="declare" name="declare" required>
        <label for="declare">I declare all details true</label><br><br>

        <input type="submit" value="Submit Application">
        <input type="reset" value="Reset">
    </form>
</body>
</html>
```

Example Explanation: Two fieldsets grouping Personal and Academic. Personal includes text with minlength, email type auto validation, tel with pattern Indian mobile, radio group gender same name, date picker, color picker. Academic includes select dropdown, datalist for state autocomplete, checkbox languages, textarea address, file upload image/* with required, and file needs post+multipart/form-data. Declaration checkbox required. This covers all form controls syllabus.

PART B: TABLES

11. Tables Introduction

Explanation: Table displays data in rows and columns. Used for tabular data like mark sheets, timetables. Must NOT be used for page layout (old practice). Structure: table container, tr row, th header bold centered, td data.

Syntax:

```
<table>
    <tr>
        <th>Header</th>
        <td>Data</td>
    </tr>
</table>
```

Important Attributes: border="1" gives visible border, caption gives title.

Example Simple:

```
<table border="1">
    <caption>Student List</caption>
    <tr>
        <th>Roll No</th>
        <th>Name</th>
        <th>Marks</th>
    </tr>
    <tr>
        <td>101</td>
        <td>Rahul</td>
        <td>85</td>
    </tr>
</table>
```

Example Explanation: table border 1 shows grid, caption Student List on top, first tr contains th headers bold centered, second tr contains td data normal left.

12. Table Structure Tags

Explanation: For professional tables use grouping tags.

Syntax & Table:

Tag	Explanation
table	Whole table container
caption	Title of table - after opening table
thead	Groups header rows
tbody	Groups body data rows
tfoot	Groups footer totals row
tr	Table Row horizontal
th	Header cell bold center
td	Data cell

Example with thead tbody tfoot:

```
<table border="1">
    <caption>Semester Marks</caption>
    <thead>
        <tr>
            <th>Code</th>
            <th>Subject</th>
            <th>Max</th>
            <th>Obtained</th>
        </tr>
```

```
    </thead>
    <tbody>
        <tr>
            <td>CS101</td>
            <td>Web Design</td>
            <td>100</td>
            <td>88</td>
        </tr>
        <tr>
            <td>CS102</td>
            <td>Data Structures</td>
            <td>100</td>
            <td>91</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="3">Total</th>
            <th>179</th>
        </tr>
    </tfoot>
</table>
```

Example Explanation: thead wraps header titles, tbody wraps data rows, tfoot wraps total row, caption on top, th bold for totals. This structure helps screen readers and printing (thead repeats on each printed page).

13. Colspan and Rowspan

Explanation: colspan merges columns horizontally, rowspan merges rows vertically. Used for complex tables like timetable where one subject spans two periods or hostel name spans multiple students.

Syntax:

```
<!-- colspan merges 2 columns -->
<td colspan="2">Web Design Lab (9am-11am)</td> <!-- Spans 2 columns -->

<!-- rowspan merges 3 rows -->
<td rowspan="3">Ganga Hostel</td> <!-- Spans 3 rows vertically -->
```

Example Colspan:

```
<table border="1">
    <caption>Colspan Timetable</caption>
    <tr>
        <th>Day / Time</th>
        <th>9am-10am</th>
        <th>10am-11am</th>
        <th>11am-12pm</th>
    </tr>
```

```
    <tr>
        <td>Monday</td>
        <td colspan="2">Web Design Lab (9am-11am) - Spans 2 columns</td>
        <td>DBMS Theory</td>
    </tr>
    <tr>
        <td>Tuesday</td>
        <td>Maths</td>
        <td>DS</td>
        <td>Web Design</td>
    </tr>
</table>
```

Example Explanation: Monday row has Day column Monday, next td colspan 2 merges 9-10 and 10-11 columns so Lab occupies 2 hours, third td DBMS. Without colspan would need two separate tds for lab.

Example Rowspan:

```
<table border="1">
    <caption>Rowspan Hostel</caption>
    <tr>
        <th>Hostel Name</th>
        <th>Student Name</th>
        <th>Room No</th>
    </tr>
    <tr>
        <td rowspan="3">Ganga Hostel</td>
        <td>Arun</td>
        <td>101</td>
    </tr>
    <tr>
        <td>Karthik</td>
        <td>102</td>
    </tr>
    <tr>
        <td>Sunil</td>
        <td>103</td>
    </tr>
    <tr>
        <td rowspan="2">Kaveri Hostel</td>
        <td>Meena</td>
        <td>201</td>
    </tr>
    <tr>
        <td>Lakshmi</td>
        <td>202</td>
    </tr>
</table>
```

Example Explanation: Ganga Hostel td rowspan 3 merges 3 rows vertically, so one hostel name appears for 3 students Arun, Karthik, Sunil. Next row for Karthik does NOT have hostel td because already merged. Similarly Kaveri Hostel rowspan 2 for 2 students.

14. Combined Complex Table - Colspan + Rowspan (10 Marks Timetable)

Explanation: Real timetable uses both. Horizontal merging for labs spanning multiple periods (colspan) and vertical merging for break (rowspan).

Syntax & Example:

```
<table border="1">
    <caption>CLASS TIME TABLE - CSE - 3rd SEM</caption>
    <thead>
        <tr>
            <th>Day</th>
            <th>9:00-10:00</th>
            <th>10:00-11:00</th>
            <th>11:00-11:15</th>
            <th>11:15-12:15</th>
            <th>12:15-1:15</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <th>Monday</th>
            <td>Web Design</td>
            <td>DBMS</td>
            <td rowspan="5">BREAK</td>
            <td colspan="2">Web Design Lab</td>
        </tr>
        <tr>
            <th>Tuesday</th>
            <td>DS</td>
            <td>Maths</td>
            <td>DBMS</td>
            <td>Library</td>
        </tr>
        <tr>
            <th>Wednesday</th>
            <td colspan="2">DBMS Lab</td>
            <td>Web Design</td>
            <td>DS Lab</td>
        </tr>
    </tbody>
</table>
```

Example Explanation: BREAK th/td rowspan 5 merges 5 rows vertically (Monday to Friday same break time). Web Design Lab colspan 2 merges 2 columns because lab 2 hours. Similarly Wednesday DBMS Lab colspan 2. First count total columns per row, ensure colspan math correct.

15. Full Mark Sheet Table with Thead Tbody Tfoot + Colspan Rowspan Combined

Explanation: Mark sheet uses rowspan for S.No/Name/Total spanning 2 header rows, colspan for Marks Obtained spanning 3 subjects.

Syntax & Example:

```
<table border="1">
    <caption>ABC College - Mark Sheet - 2026</caption>
    <thead>
        <tr>
            <th rowspan="2">S.No</th>
            <th rowspan="2">Student Name</th>
            <th colspan="3">Marks Obtained</th>
            <th rowspan="2">Total</th>
        </tr>
        <tr>
            <th>Web Design</th>
            <th>DBMS</th>
            <th>Data Structure</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Rahul Kumar</td>
            <td>85</td>
            <td>78</td>
            <td>90</td>
            <td>253</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Priya S</td>
            <td>92</td>
            <td>88</td>
            <td>95</td>
            <td>275</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th colspan="5">Class Average</th>
            <th>264</th>
        </tr>
    </tfoot>
</table>
```

Example Explanation: First header row has S.No rowspan 2 so spans 2 rows vertically, Marks Obtained colspan 3 spans 3 subjects horizontally, Total rowspan 2. Second header row has 3 subjects. Footer Class Average colspan 5 merges 5 columns for label and average in last th.

16. Forms + Tables Combined - Old Method

Explanation: Historically forms were laid using tables for alignment - labels in one td, inputs in another.

Syntax & Example:

```
<h2>Login Form using Table for Layout (Traditional)</h2>
<form action="login.php" method="post">
    <table border="1">
        <caption>Login</caption>
        <tr>
            <td><label for="user">Username:</label></td>
            <td><input type="text" id="user" name="user" required></td>
        </tr>
        <tr>
            <td><label for="pass">Password:</label></td>
            <td><input type="password" id="pass" name="pass" required></td>
        </tr>
        <tr>
            <td colspan="2"><input type="submit" value="Login"></td>
        </tr>
    </table>
</form>
```

Example Explanation: Table used for form layout - each row has label td and input td, last row colspan 2 merges for submit button center. Modern practice uses fieldset/div not table for layout, but exam may still ask difference and this method.

Conclusion: Forms theory must cover action/method/enctype, GET vs POST tabular difference, input types with syntax, other controls select/datalist/fieldset, validation attributes required/pattern/min/max, difference readonly vs disabled. Tables must cover tr/th/td/caption/thead/tbody/tfoot, colspan/rowspan merging logic, and complex mark sheet/timetable programs. Together they form interactive and data presentation layers of HTML5.

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

2026

Year:

I

Semester:

I

Unit:

I

Topic:

Accessibility and SEO - Browser DevTools and Emmet

PART A: ACCESSIBILITY

1. What is Accessibility?

Explanation: Accessibility (a11y - a + 11 letters + y) is practice of making websites usable for ALL people including disabled (blind, deaf, motor disability, color blind). Blind uses Screen Reader (NVDA, JAWS) that reads page aloud, so images need alt text, deaf needs captions for video, motor disability needs keyboard navigation without mouse.

Syntax / Principle Concept:

```
Accessible Website = Semantic HTML + Alt Text + Labels + Keyboard Navigation +
ARIA
WCAG Principles - POUR:
P - Perceivable, O - Operable, U - Understandable, R - Robust
```

Example: A blind user visits college website. If logo `<img src="logo.jpg">` has no alt, screen reader says "image" only, no info. If `<img src="logo.jpg" alt="ABC College Logo">` then screen reader says "ABC College Logo".

Example Explanation: Accessibility ensures screen reader can announce content. Alt text, proper headings, labels make site perceivable and operable.

---

2. WCAG Principles - POUR

Explanation: WCAG (Web Content Accessibility Guidelines) by W3C sets standards. 4 principles POUR must remember for exam.

Syntax / Table:

Principle	Meaning	HTML Syntax Example
P - Perceivable	Info must be perceivable by senses	`alt` for images, `<track>` for video captions
O - Operable	Interface must be operable via keyboard	`tabindex="0"`, keyboard nav, skip link
U - Understandable	Content must be understandable	`lang="en"`, clear labels
R - Robust	Works with all browsers & assistive tech	Valid HTML5, semantic tags, ARIA

Example: Perceivable: `<img alt="Chart 80% passed">` helps blind perceive. Operable: `tabindex` allows keyboard focus.

Example Explanation: If site fails any POUR, disabled users blocked. WCAG levels: A (basic), AA (standard - most follow), AAA (highest).

---

3. Accessibility in HTML - Practical Features

A) Language, Title, Headings, Skip Link

Explanation: lang attribute tells screen reader language pronunciation. title is first announced. Headings hierarchy h1->h2->h3 helps screen reader navigate via heading list. Skip to main content link helps keyboard users bypass repeated nav.

Syntax:

```
<html lang="en">
<title>Accessible Page - Good Example</title>
<a href="#maincontent">Skip to Main Content</a>
<main id="maincontent">Main content</main>
```

Example Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Accessible Page</title>
</head>
<body>
    <a href="#maincontent">Skip to Main Content</a>
    <header><h1>ABC College Website</h1></header>
    <nav><ul><li><a href="index.html">Home</a></li></ul></nav>
    <main id="maincontent">
        <h2>Welcome Page</h2>
        <p>Screen reader reads easily.</p>
    </main>
</body>
</html>
```

Example Explanation: lang en for English pronunciation. Skip link href="#maincontent" jumps to main id maincontent, so keyboard user pressing Tab first sees skip link and can jump over nav directly to main content, saving many Tab presses.

B) Alt Text, Label, Fieldset, Figure, Table Scope

Explanation: Alt for images, Label for inputs (for/id linking), Fieldset/Legend groups form controls, Figure/Figcaption groups image+caption, scope in th (col/row) associates header with data cells for screen reader.

Syntax Table:

Feature	Syntax	Explanation
Alt	`<img src="a.jpg" alt="description">`	Screen reader reads alt, SEO uses alt
Label	`<label for="email">Email:</label><input id="email">`	Clicking label focuses input, screen reader reads label
Fieldset	`<fieldset><legend>Personal Info</legend>... </fieldset>`	Group + title for screen reader
Figure	`<figure><img...> <figcaption>Caption</figcaption></figure>`	Semantic grouping
Table Scope	`<th scope="col">Name</th><th scope="row">101</th>`	Col header, row header association

Example:

```
<img src="college_building.jpg" alt="Front view of ABC College Main Building
with 3 floors" width="400">
<img src="divider.jpg" alt="" aria-hidden="true"> <!-- Decorative - Empty alt
so skip -->

<label for="email">Email:</label>
<input type="email" id="email" name="email">

<fieldset>
    <legend>Gender</legend>
    <input type="radio" id="m" name="gender"><label for="m">Male</label>
</fieldset>

<table border="1">
    <tr><th scope="col">Roll No</th><th scope="col">Name</th></tr>
    <tr><th scope="row">101</th><td>Rahul</td></tr>
</table>
```

Example Explanation: First image informative alt detailed, second decorative alt="" so screen reader ignores. Label for email linked to input id email. Fieldset legend Gender read for each radio by screen reader. Table th scope col means column header, scope row means row header, helps screen reader announce "Roll No: 101, Name: Rahul".

---

4. ARIA - Accessible Rich Internet Applications

Explanation: ARIA provides extra accessibility info when native HTML not enough. Used for custom widgets made with div. Attributes start with aria-.

Syntax & Most Important ARIA:

ARIA Attribute	Syntax	Explanation
aria-label	`<button aria-label="Close Menu">X</button>`	Label when no visible text (icon button)
aria-labelledby	`<form aria-labelledby="formTitle"><h2 id="formTitle">Form</h2></form>`	Uses another element as label
aria-hidden	`<img alt="" aria-hidden="true">`	Hide decorative from screen reader
aria-required	`<input aria-required="true" required>`	Required for screen reader
aria-describedby	`<input aria-describedby="help"><p id="help">Min 8 chars</p>`	Extra description referenced
role	`<div role="navigation">`, `role="alert"`, `role="main"`	Defines role of element

Example Programs:

```
<!-- aria-label for icon button -->
<button aria-label="Close Menu">X</button>
<!-- Screen reader reads Close Menu instead of just X -->

<!-- aria-labelledby -->
<h2 id="formTitle">Admission Form</h2>
<form aria-labelledby="formTitle"></form>
<!-- Screen reader will read Admission Form as form label -->

<!-- aria-hidden for decorative -->
<img src="decoration.jpg" alt="" aria-hidden="true">
<!-- Completely skipped by screen reader -->

<!-- aria-describedby extra help -->
<label for="pwd">Password:</label>
<input type="password" id="pwd" aria-describedby="pwdHelp">
<p id="pwdHelp">Min 8 chars, 1 uppercase, 1 number</p>
<!-- Screen reader reads label + description pwdHelp when focusing input -->

<!-- role -->
<div role="banner">Header</div>
<div role="navigation"><a href="#">Home</a></div>
<div role="main">Main Content</div>
<div role="alert">Form submitted successfully!</div>
```

Example Explanation: Rule: Use native semantic first `<nav> <main>`, only if not possible use ARIA role. aria-label needed for icon buttons without text. aria-describedby gives additional hint like password rules. aria-hidden true + alt empty hides decorative completely.

---

5. Accessible Forms, Tables, Media - Full Programs

A) Accessible Form

Explanation: Every input must have label, required fields marked with aria-required, grouped with fieldset/legend, help text via aria-describedby.

Syntax & Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Accessible Form</title>
</head>
<body>
    <h1>Accessible Registration Form</h1>
    <form action="submit.php" method="post">
```

```
        <fieldset>
            <legend>Personal Information</legend>

            <label for="fullname">Full Name: *</label>
            <input type="text" id="fullname" name="fullname" required aria-
required="true" placeholder="Enter full name"><br><br>

            <label for="email_acc">Email:</label>
            <input type="email" id="email_acc" name="email" required aria-
describedby="emailHelp">
            <p id="emailHelp">We will never share your email</p><br>

            <label>Choose Course (Required)</label>
            <input type="radio" id="bca" name="course" value="bca" required>
            <label for="bca">BCA</label>
        </fieldset>
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Example Explanation: Fieldset legend Personal Information groups fields, screen reader announces legend. Each input has label for/id pairing. Full Name has required + aria-required true + * visual star but * aria-hidden for screen reader not needed. Email has aria-describedby emailHelp which gives extra note. All requirements for accessible form.

B) Accessible Table with Scope

Explanation: scope attribute helps screen reader associate header with data.

Syntax & Example:

```
<table border="1">
    <caption>Student Marks</caption>
    <thead>
        <tr>
            <th scope="col">Roll No</th>
            <th scope="col">Name</th>
            <th scope="col">Web Design</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <th scope="row">101</th>
            <td>Rahul Kumar</td>
            <td>88</td>
        </tr>
    </tbody>
</table>
```

Example Explanation: scope col for column headers, scope row for row header Roll No 101. Screen reader will announce "Roll No: 101, Name: Rahul Kumar, Web Design: 88" row wise, understanding association.

C) Accessible Images and Video Captions

Explanation: Images need meaningful alt, decorative alt empty, video needs captions via track for deaf.

Syntax & Example:

```
<!-- Good alt -->
<img src="college_building.jpg" alt="Front view of ABC College Main Building
with 3 floors" width="400">

<!-- Decorative empty -->
<img src="divider.jpg" alt="" aria-hidden="true">

<!-- Complex image with figure -->
<figure>
    <img src="chart.jpg" alt="Bar chart shows 80% students passed">
    <figcaption>Detailed: In 2025, 80% passed, 20% failed.</figcaption>
</figure>

<!-- Accessible Video -->
<video controls width="400">
    <source src="lecture.mp4" type="video/mp4">
    <track src="captions_en.vtt" kind="captions" srclang="en" label="English
Captions" default>
</video>
```

Example Explanation: First image good alt detailed. Second decorative alt empty + aria-hidden true skipped. Figure with figcaption provides detailed description for complex chart. Video track kind captions provides subtitles for deaf, default shows automatically.

PART B: SEO

6. What is SEO?

Explanation: SEO = Search Engine Optimization. Process to rank higher on Google/Bing. Higher rank = More visitors. Three types: On-Page (inside HTML - title, meta, headings, alt), Off-Page (outside - backlinks, social shares), Technical (speed, mobile friendly, HTTPS).

Syntax / How Google Works:

```
1. Crawling: Google Bot visits pages and reads HTML
2. Indexing: Stores pages in database
3. Ranking: Shows most relevant first based on 200+ factors (title, content,
speed, mobile)
```

Example: User searches "web design course Chennai". If your page title contains that keyword and has good content, Google ranks higher.

Example Explanation: SEO makes your site discoverable. Without SEO, even good site may be on page 10 of Google, no one visits.

---

7. On-Page SEO with HTML Tags - Detailed

Explanation: On-Page SEO done inside HTML page itself. Most important for theory exam.

A) Title Tag

Explanation: Most important SEO tag, 50-60 chars, includes main keyword, unique per page, shown as clickable headline in Google.

Syntax: `<title>Keyword - Descriptive Title | Brand</title>`

Example:

```
<!-- BAD -->
<title>Home</title>

<!-- GOOD SEO -->
<title>Learn Web Design Course in Chennai - ABC College | Best CSE
Course</title>
```

Example Explanation: Bad title Home has no keyword, bad ranking. Good title includes primary keyword web design course chennai, brand ABC College, compelling.

B) Meta Description

Explanation: 150-160 chars summary shown below title in Google snippet. Include keyword naturally + call to action. Not direct ranking factor but improves click rate.

Syntax: `<meta name="description" content="150-160 chars summary with keyword and CTA">`

Example:

```
<meta name="description" content="Best Web Design Course in Chennai at ABC
College. Learn HTML5, CSS3, JavaScript, Full Stack. Admission open 2026. 100%
placement support.">
```

Example Explanation: Description contains keyword web design course Chennai, benefits HTML5 CSS3 Full Stack, CTA admission open and placement support to encourage click.

C) Meta Viewport (Compulsory for Mobile SEO)

Explanation: Without viewport meta, mobile shows desktop zoomed out, poor UX, Google penalizes.

Syntax: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

Example Explanation: width=device-width makes page width equal to device width, initial-scale 1.0 no zoom. Mandatory for responsive and Mobile First Indexing.

D) Headings

Explanation: Use one h1 per page with main keyword, then h2 h3 hierarchy. Google gives more weight to h1.

Syntax & Example:

```
<!-- BAD - No keyword, multiple h1 vague -->
<h1>Welcome</h1>

<!-- GOOD -->
<h1>HTML5 Semantic Elements Complete Notes for Semester Exam</h1>
<h2>Why Use Semantic Tags?</h2>
```

Example Explanation: Good h1 includes main keyword HTML5 Semantic Elements Notes, clear topic.

E) Image SEO

Explanation: File name descriptive with keyword, alt with keyword naturally.

Syntax & Example:

```
<!-- BAD -->
<img src="img1.jpg" alt="image">

<!-- GOOD SEO -->
<img src="html5-semantic-layout-diagram.jpg" alt="HTML5 Semantic Layout Diagram
showing header nav main article aside footer">
```

Example Explanation: Good file name includes keyword semantic layout diagram, alt detailed with keywords header nav main etc. Helps image search ranking.

F) URL Structure & Internal Linking

Explanation: Short descriptive SEO friendly URL with hyphens, not id.

Syntax & Example:

```
<!-- BAD URL -->
https://example.com/page?id=123

<!-- GOOD URL -->
https://example.com/courses/web-design

<!-- BAD Anchor -->
```

```
<a href="page2.html">Click here</a>

<!-- GOOD Anchor descriptive -->
<a href="html5-notes.html">Read Complete HTML5 Forms and Validation Notes</a>
```

Example Explanation: Good URL contains keyword courses/web-design, easy to read. Good anchor text describes destination with keywords, helps SEO, screen reader also benefits.

G) Canonical Tag

Explanation: Avoids duplicate content penalty when same content available via multiple URLs.

Syntax: `<link rel="canonical" href="https://www.abccollege.edu.in/courses/web-design.html">`

Example Explanation: If web-design page accessible via both /web-design.html and /courses/web-design.html, canonical tells Google preferred URL is second one, consolidating ranking.

H) Open Graph & Favicon

Explanation: Open Graph controls how page looks when shared on Facebook/WhatsApp. Favicon small icon in tab.

Syntax & Example:

```
<meta property="og:title" content="Web Design Course - ABC College">
<meta property="og:description" content="Join Best Web Design Course in
Chennai">
<meta property="og:image"
content="https://abccollege.edu.in/images/webdesign.jpg">
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

Full SEO Head Example - Most Important for 10 Marks:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web Design Course in Chennai - Learn HTML5 | ABC College</title>
    <meta name="description" content="Best Web Design Course in Chennai at ABC
College. Learn HTML5, Semantic Tags, Forms, Accessibility and SEO. Admissions
2026 open.">
    <meta name="keywords" content="web design course, html5 course chennai, bca
college chennai">
    <meta name="author" content="ABC College">
    <meta name="robots" content="index, follow">
    <link rel="canonical" href="https://abccollege.edu.in/web-design-
course.html">
    <meta property="og:title" content="Web Design Course - ABC College">
```

```
    <meta property="og:description" content="Join Best Web Design Course in
Chennai">
    <meta property="og:image"
content="https://abccollege.edu.in/images/webdesign.jpg">
    <link rel="icon" href="favicon.ico">
</head>
<body>
    <h1>Web Design Course in Chennai</h1>
</body>
</html>
```

Example Explanation: This head includes all SEO tags: title with keywords, viewport mandatory, description 150-160 chars with keyword + CTA, keywords, author, robots index follow allows crawling, canonical preferred URL, Open Graph for social sharing, favicon.

PART C: BROWSER DEVTOOLS

8. What is Browser DevTools?

Explanation: DevTools are built-in debugging tools in Chrome, Firefox, Edge to inspect, debug, test website without extra software.

Syntax / How to Open:

- Windows: `F12` or `Ctrl + Shift + I` or Right Click → Inspect
- Mac: `Cmd + Opt + I`
- Inspect Element mode: `Ctrl + Shift + C`
- Console directly: `Ctrl + Shift + J`

Example: Press F12 on any page, Elements panel shows DOM.

Example Explanation: DevTools lets you edit HTML live, check CSS, see errors, test responsive.

9. DevTools Panels - Theory

Explanation: Different panels for different purposes.

Syntax Table:

Panel	Explanation	Use
Elements	Live HTML & DOM tree	Edit HTML instantly, check alt missing, copy selector
Console	JS errors, warnings	Shows [Accessibility] Images should have alt, run JS commands
Sources	All files	See HTML/CSS/JS files, debug JS
Network	All files loaded	Check images, videos loading time, 404 errors
Application	Storage	Cookies, Local Storage, Cache, Favicon
Lighthouse	Audit tool	Generates score 0-100 for Performance, Accessibility, SEO

Example: In Elements panel double click tag text to edit live. In Console `$0` shows currently selected element from Elements panel.

Example Explanation: If image src wrong, Network shows 404 red, Console shows error. Elements shows live DOM after JS modifications.

10. How to Check Accessibility & SEO using DevTools

Explanation: Lighthouse panel provides automated audit.

Syntax / Steps:

```
1. F12 → Elements → Right panel → Accessibility Tab → Shows Accessibility Tree,
Role, Name
2. F12 → Lighthouse Tab → Check Accessibility → Generate Report → Score 0-100
   Checks: alt, label, heading order, contrast, aria-* validity
3. F12 → Lighthouse → Check SEO → Generate Report
   Checks: Title exists?, Meta Description?, Viewport?, Crawlable?, Alt?,
Descriptive links?
```

Example Program for Testing:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Test My Page in DevTools</title>
    <meta name="description" content="This is test page to check DevTools">
</head>
<body>
    <h1>Open F12 and Test</h1>
    <p>Steps: F12 → Lighthouse → Accessibility + SEO → Analyze</p>
    <img src="test.jpg" alt="Test image for accessibility check">
    <form>
        <label for="testInput">Name:</label>
        <input type="text" id="testInput" name="testInput" required>
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Example Explanation: This page has proper title, viewport, description, alt, label, required. Lighthouse will give 90+ score. If you remove alt, Console shows warning [Accessibility] Images should have alt attribute and Lighthouse score drops.

PART D: EMMET

11. What is Emmet?

Explanation: Emmet is plugin built into VS Code that converts short abbreviations into full HTML code. Saves 80% typing time. Instead of typing 15 lines for boilerplate, type `!` + Tab generates full skeleton.

Syntax: Type abbreviation → Press Tab or Enter.

Example: Type `!` + Tab → Expands to full HTML5 skeleton.

Example Explanation: Emmet is like shorthand for HTML.

12. Emmet Operators - Syntax

Explanation: Operators define relationship.

Table:

Operator	Meaning	Syntax Example	Expanded Example
`>`	Child (Inside)	`div>p`	`<div><p></p></div>`
`+`	Sibling (Next)	`div+p`	`<div></div><p></p>`
`^`	Climb up	`div>p>span^a`	`span` inside p, `a` sibling of p inside div
`*`	Multiplication	`ul>li*5`	`ul` with 5 `li`
`()`	Grouping	`(div>p)*2`	2 groups of div>p
`#`	ID	`div#header`	`<div id="header">`
`.`	Class	`div.box`	`<div class="box">`
`[]`	Attributes	`a[href="about.html"]`	`<a href="about.html">`
`{}`	Text	`p{Hello}`	`<p>Hello</p>`
`$`	Numbering	`li*3{Item $}`	Item 1, Item 2, Item 3
`lorem`	Dummy text	`lorem10`	10 dummy words

13. Emmet Most Important Examples - Syntax + Example + Explanation

A) Boilerplate:

Syntax: `!` + Tab or `html:5`

Example Expansion:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
```

```
</head>
<body>

</body>
</html>
```

Example Explanation: Single character ! generates 11 lines of HTML5 skeleton. Saves time in exam.

B) ID, Class, Attributes, Text:

Syntax & Examples:

```
<!-- Syntax: div#header -->
<div id="header"></div>
<!-- Explanation: # = ID selector -->

<!-- Syntax: div.container -->
<div class="container"></div>
<!-- Explanation: . = class -->

<!-- Syntax: a[href="about.html" title="About Us"] -->
<a href="about.html" title="About Us"></a>
<!-- Explanation: [] for attributes -->

<!-- Syntax: p{Hello World} -->
<p>Hello World</p>
<!-- Explanation: {} for text content -->

<!-- Syntax: img[src="logo.jpg" alt="College Logo" width="200"] -->
<img src="logo.jpg" alt="College Logo" width="200">
<!-- Explanation: Multiple attributes inside [] -->
```

C) Child > and Sibling +:

Syntax & Example:

```
<!-- Syntax: ul>li -->
<ul>
    <li></li>
</ul>
<!-- Explanation: > = li is child inside ul -->

<!-- Syntax: nav>ul>li*3>a -->
<nav>
    <ul>
        <li><a href=""></a></li>
        <li><a href=""></a></li>
        <li><a href=""></a></li>
    </ul>
</nav>
```

```
<!-- Explanation: nav child ul child 3 li child a - Creates navigation menu
quickly -->

<!-- Syntax: header+main+footer -->
<header></header>
<main></main>
<footer></footer>
<!-- Explanation: + = siblings side by side -->
```

D) Multiplication * and Numbering $:

Syntax & Examples:

```
<!-- Syntax: ul>li*5 -->
<ul>
    <li></li>
    <li></li>
    <li></li>
    <li></li>
    <li></li>
</ul>
<!-- Explanation: *5 repeats 5 times -->

<!-- Syntax: ul>li*5{Item $} -->
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
    <li>Item 4</li>
    <li>Item 5</li>
</ul>
<!-- Explanation: $ = auto numbering 1,2,3. Item $ inserts number -->

<!-- Syntax: h$*6 -->
<h1></h1>
<h2></h2>
<h3></h3>
<h4></h4>
<h5></h5>
<h6></h6>
<!-- Explanation: $ in tag name generates h1 to h6 -->
```

E) Grouping () and Table:

Syntax & Example:

```
<!-- Syntax: (header>nav)+main+footer -->
<header>
    <nav></nav>
</header>
```

```
<main></main>
<footer></footer>
<!-- Explanation: () groups header>nav as one unit, then siblings main footer -
->

<!-- Syntax: table[border="1"]>(tr>th*3)+(tr*3>td*3) -->
<table border="1">
    <tr><th></th><th></th><th></th></tr>
    <tr><td></td><td></td><td></td></tr>
    <tr><td></td><td></td><td></td></tr>
    <tr><td></td><td></td><td></td></tr>
</table>
<!-- Explanation: Table border 1, first row tr>th*3 three headers, plus ( )
grouping second part tr*3>td*3 three rows three columns data - Creates full
table quickly -->
```

F) Form and Lorem:

Syntax & Examples:

```
<!-- Syntax: form:post -->
<form action="" method="post"></form>
<!-- Explanation: form:post shortcut = post method form -->

<!-- Syntax: input:email -->
<input type="email" name="" id="">

<!-- Syntax: select>option*3 -->
<select><option></option><option></option><option></option></select>

<!-- Syntax: p>lorem10 -->
<p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Natus, iusto.</p>
<!-- Explanation: lorem10 generates 10 dummy words inside p -->
```

14. Complete Page via One Line Emmet

Explanation: Full page structure can be built in single abbreviation line.

Syntax - Abbreviation:

```
header>h1{ABC College}+nav>ul>li*4>a{Link
$}^+main>section>h2{Welcome}+p>lorem20^+aside>h3{News}+ul>li*2{News
$}^+footer>p{Copyright 2026}
```

Example Expansion - After Tab:

```
<header>
    <h1>ABC College</h1>
    <nav>
        <ul>
            <li><a href="">Link 1</a></li>
            <li><a href="">Link 2</a></li>
            <li><a href="">Link 3</a></li>
            <li><a href="">Link 4</a></li>
        </ul>
    </nav>
</header>
<main>
    <section>
        <h2>Welcome</h2>
        <p>Lorem ipsum dolor sit amet consectetur...</p>
    </section>
</main>
<aside>
    <h3>News</h3>
    <ul>
        <li>News 1</li>
        <li>News 2</li>
    </ul>
</aside>
<footer>
    <p>Copyright 2026</p>
</footer>
```

Example Explanation: This single line abbreviation uses > child, + sibling, * multiplication, {} text, $ numbering, ^ climb up (^+ means climb up one level then sibling aside). One line generates entire semantic page layout that would normally need 30+ lines typed manually. This demonstrates power of Emmet for quick prototyping in exam or project.

PART E: COMBINED FINAL PROGRAM

15. Perfect Page - Accessible + SEO Friendly + Emmet Generated + DevTools Verifiable

Explanation: Ideal model for 10 marks combining all concepts.

Syntax & Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web Design Course in Chennai - Learn HTML5 | ABC College</title>
    <meta name="description" content="Join Best Web Design Course in Chennai at
ABC College. Learn HTML5, Semantic Tags, Forms, Accessibility and SEO.
```

```
Admissions 2026 open.">
    <meta name="author" content="ABC College">
    <meta name="robots" content="index, follow">
    <link rel="canonical" href="https://abccollege.edu.in/web-design-
course.html">
    <meta property="og:title" content="Web Design Course - ABC College">
    <meta property="og:description" content="Join Best Web Design Course in
Chennai">
    <meta property="og:image"
content="https://abccollege.edu.in/images/webdesign.jpg">
    <link rel="icon" href="favicon.ico">
</head>
<body>
    <a href="#main-content">Skip to Main Content</a>
    <header role="banner">
        <h1>ABC College - Web Design Course Chennai</h1>
    </header>
    <nav role="navigation" aria-label="Main Navigation">
        <ul>
            <li><a href="index.html" title="Go to Home Page">Home</a></li>
            <li><a href="about.html" title="About ABC College">About Us</a>
</li>
        </ul>
    </nav>
    <main id="main-content" role="main">
        <article>
            <h2>Why Learn HTML5 Semantic Elements?</h2>
            <p>HTML5 semantic elements help in <strong>SEO</strong> and
<strong>Accessibility</strong>.</p>
            <figure>
                <img src="html5-semantic-tags-diagram.jpg" alt="Diagram showing
header, nav, main, article, aside, footer semantic layout" width="500">
                <figcaption>Fig 1: HTML5 Semantic Layout helps SEO and
Accessibility</figcaption>
            </figure>
            <form aria-labelledby="formHeading">
                <h4 id="formHeading">Apply Now</h4>
                <label for="studentName">Full Name:</label>
                <input type="text" id="studentName" name="studentName" required
aria-required="true">
            </form>
        </article>
    </main>
    <footer role="contentinfo">
        <p>&copy; 2026 ABC College, Meenambakkam, Chennai.</p>
        <address>Contact: <a
href="mailto:info@abccollege.edu.in">info@abccollege.edu.in</a></address>
    </footer>
</body>
</html>
```

Example Explanation: This final program demonstrates:

- Accessibility: lang, skip link href="#main-content", semantic header/nav/main/footer, role banner/navigation/main/contentinfo, alt detailed, label for/id, aria-label, aria-required, address.
- SEO: title with keyword web design course chennai, viewport, description 150 chars with keyword+CTA, robots index follow, canonical, Open Graph og:title/og:description/og:image, favicon, h1 with keyword, figure alt with keywords, descriptive anchor text.
- DevTools: Open F12 → Lighthouse → Accessibility + SEO → Generate Report will give 90+ score because all best practices followed. Console will show no warnings about alt missing.
- Emmet: This whole page could be generated quickly using Emmet abbreviation: header>h1{ABC College}+nav>ul>li*2>a^ + main#main-content>article>h2+p+figure>img+figcaption^^form>label+input + footer>p+address - Press Tab to expand quickly.

Conclusion: Accessibility via semantic HTML + ARIA + alt + label + scope ensures inclusivity per WCAG POUR. SEO via title + meta description + viewport + headings + alt + canonical + OG ensures discoverability and higher ranking. DevTools Elements + Lighthouse panels help verify and audit both scores. Emmet operators > + * # . [] {} $ provide rapid code generation. Together they form modern professional workflow beyond basic HTML and are essential for high quality website creation.